Welcome to CSC331 Fall 2025 Data Structures

Professor Kevin Byron (kbyron@bmcc.cuny.edu)
Department of CIS, Room F-1030-I
Office hours: Mon 2pm-5pm (in-person and Zoom)
Office phone: 212-220-1490
Borough of Manhattan Community College/CUNY

CSC331 Fall 2025
Data Structures

Week 4:

Stack data structure
Polish notation for arithmetic expression
Program #3 assignment
User-defined vs Standard Template Library (STL) stack class definition

Professor Kevin Byron
Department of CIS
Borough of Manhattan Community College/CUNY

Reminders

- Program 1 due date
  - 1159pm Fri 9-19-2025
  - late submission receives no credit
- No class Mon 9-22-2025 thru Wed 9-24-2025, Wed 10-1-2025, Thu 10-2-2025
- Exam 1
  - Section 501 Tue 9-30-2025 – Zoom 630pm-730pm
  - Section 172 Wed 10-8-2025 – In person room M-1001 630pm-730pm
    - Closed book, closed notes, no devices
  - Section 500 Wed 10-8-2025 – In person room N-453 1010am-1110am
    - Closed book, closed notes, no devices
- Program 2 due date
  - 1159pm Fri 10-10-2025
  - late submission receives no credit
- Program 3 due date
  - 1159pm Fri 10-31-2025
  - late submission receives no credit
- Posted on Brightspace
  - Week 4 lecture slides, C++ stack with user defined class (chap1a.cpp), C++ stack with STL (chap1c.cpp), program 3 specifications, stack331.h header file, practice quiz 1

Stacks

- A stack is a list of elements in which the addition and deletion of elements occurs only at one end, called the top of the stack.
- In a cafeteria, the second tray in a stack of trays can be removed only if the first tray has been removed.
- The elements at the bottom of the stack have been in the stack the longest.
- The top element of the stack is the last element added to the stack.
- A stack is called a Last In First Out (LIFO) data structure.

Stacks

- The add operation, called push, adds an element onto a stack.
- The top operation retrieves the top element of the stack.
- The pop operation removes the top element from the stack.

Stacks

- To implement a a stack these operations are used:
  - **`initializeStack`**—Initializes the stack to an empty state.
  - **`isEmptyStack`**—Determines whether the stack is empty. If the stack is empty, it returns the value `true`; otherwise, it returns the value `false`.
  - **`isFullStack`**—Determines whether the stack is full. If the stack is full, it returns the value `true`; otherwise, it returns the value `false`.
  - **`push`**—Adds a new element to the top of the stack. The input to this operation consists of the stack and the new element. Prior to this operation, the stack must exist and must not be full.
  - **`top`**—Returns the top element of the stack. Prior to this operation, the stack must exist and must not be empty.
  - **`pop`**—Removes the top element of the stack. Prior to this operation, the stack must exist and must not be empty.

Stacks

- Application of stacks: expression evaluation

In the early 1920s, the Polish mathematician Jan Lukasiewicz discovered that if operators were written before the operands (**prefix** or **Polish** notation; for example, + *a b*), the parentheses can be omitted. In the late 1950s, the Australian philosopher and early computer scientist Charles L. Hamblin proposed a scheme in which the operators *follow* the operands (postfix operators), resulting in the **Reverse Polish** notation. This has the advantage that the operators appear in the order required for computation.

For example, the expression:

$a + b \star c$

in a postfix expression is:

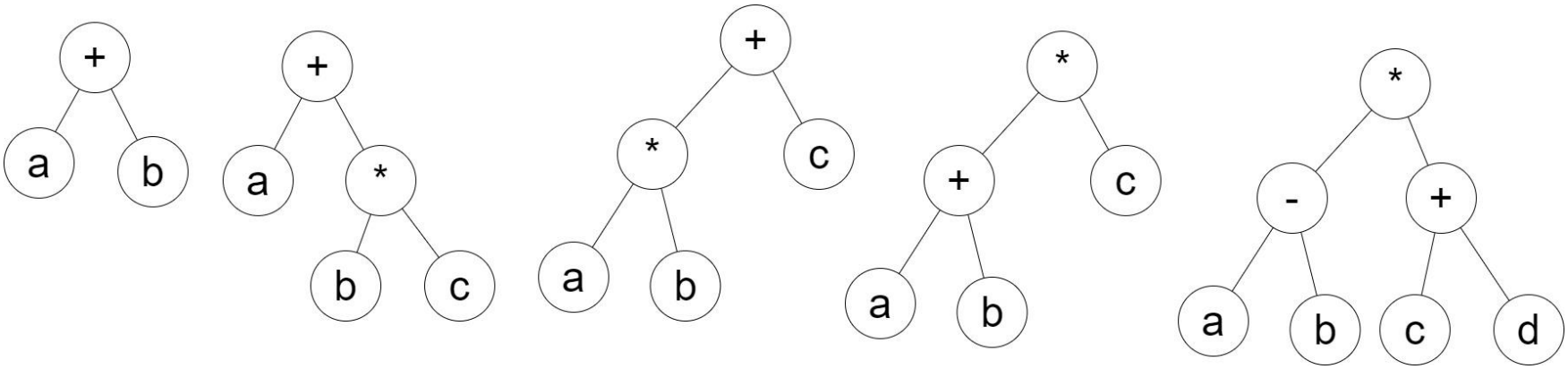
$a\ b\ c \star +$

Stacks

- Expression examples

Infix expression	Equivalent postfix expression
$a + b$	$a\ b\ +$
$a + b * c$	$a\ b\ c\ *\ +$
$a * b + c$	$a\ b\ *\ c\ +$
$(a + b) * c$	$a\ b\ +\ c\ *$
$(a - b) * (c + d)$	$a\ b\ -\ c\ d\ +\ *$
$(a + b) * (c - d / e) + f$	$a\ b\ +\ c\ d\ e\ /\ -\ *\ f\ +$

- Equivalent binary tree models of first 5 above



Stacks

- Stack processing with user defined class

```
// chap1a.cpp
// kbyron@bmcc.cuny.edu
// 3/3/19
// C program to demonstrate user-defined stack class
#include <cmath>
#include <iostream>
using namespace std;
class stackType{
  public:
    bool empty();
    void pop();
    void push(int num);
    int top();
    stackType();
  private:
    struct nodeType{
      int info;
      nodeType *link;
    };
    nodeType *head,*temp;
};
```

```
int main(){
  cout<<"we will use stack functions."<<endl;
  stackType stack;
  stack.push(7);
  cout<<stack.top()<<endl;
  for(int i=0;i<7;i++)
    stack.push(pow(2,i));
  while(!stack.empty()){
    cout<<stack.top()<<" ";
    stack.pop();
  }
  cout<<endl;
  cout<< "done."<<endl;
  return 0;
}
```

Stacks

- Stack processing with user defined class

```
bool stackType::empty(){
  return(head == NULL);
}
void stackType::pop(){
  if(head == NULL)
    cout << "pop() function failed ... empty stack.\n";
  else{
    temp = head;
    head = head->link;
    delete temp;
  }
}
void stackType::push(int num){
  temp = new nodeType;
  temp->link = head;
  temp->info = num;
  head = temp;
}
int stackType::top(){
  if(head == NULL)
    cout << "top() function failed ... empty stack.\n";
  else
    return head->info;
}
stackType::stackType(){ //default constructor
  head = NULL;
}
```

Stacks

- Stack processing with standard class

```
// chap1c.cpp
// kbyron@bmcc.cuny.edu
// 3/3/19
// C program to demonstrate stack standard class
#include <cmath>
#include <iostream>
#include <stack>
using namespace std;
int main(){
  cout<<"we will use stack functions."<<endl;
  stack<int> stack;
  stack.push(7);
  cout<<stack.top()<<endl;
  for(int i=0;i<7;i++)
    stack.push(pow(2,i));
  while(!stack.empty()){
    cout<<stack.top()<<" ";
    stack.pop();
  }
  cout<<endl;
  cout<< "done."<<endl;
  return 0;
}
```

Stacks

Evaluate the following postfix expressions:

a. `8 2 + 3 * 16 4 / - =`

b. `12 25 5 1 / / * 8 7 + - =`

c. `70 14 4 5 15 3 / * - - / 6 + =`

d. `3 5 6 * + 13 - 18 2 / + =`

Convert the following infix expressions to postfix notations:

a. `(A + B) * (C + D) - E`

b. `A - (B + C) * D + E / F`

c. `((A + B) / (C - D) + E) * F - G`

d. `A + B * (C + D) - E / F * G + H`

Write the equivalent infix expression for the following postfix expressions:

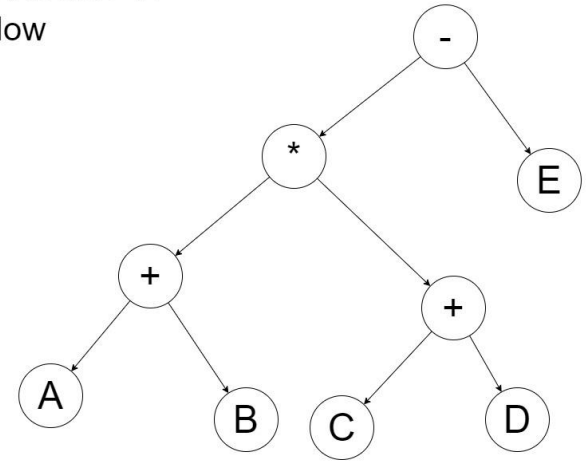
a. `A B * C +`

b. `A B + C D - *`

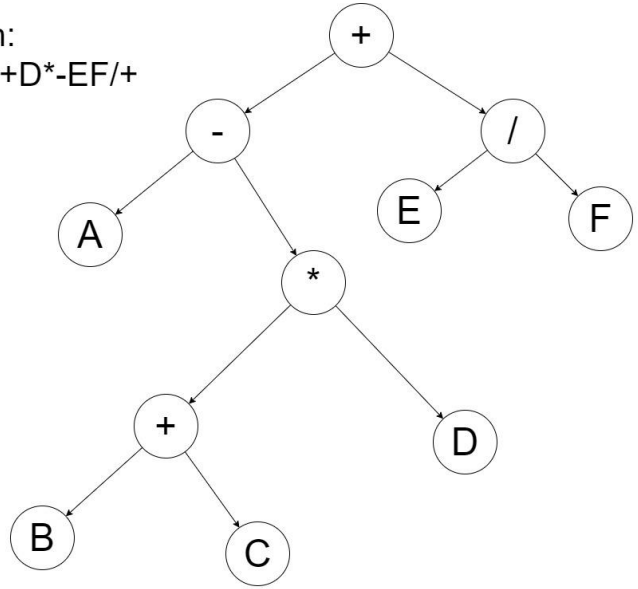
c. `A B - C - D *`

Stacks – Infix to Postfix Solutions

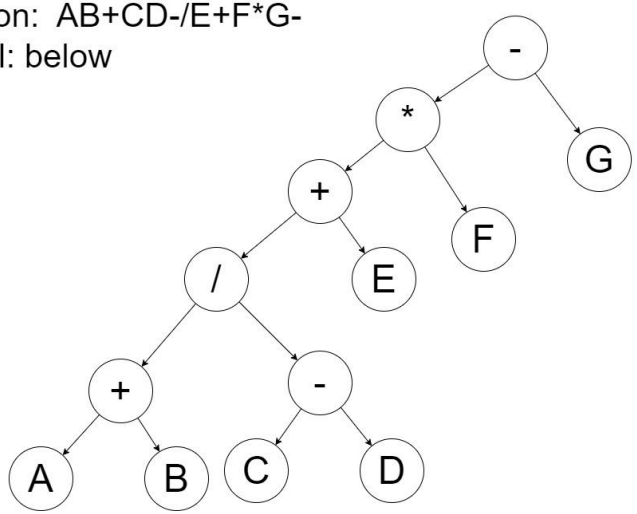
2a conversion:
solution: AB+CD+*E-
model: below



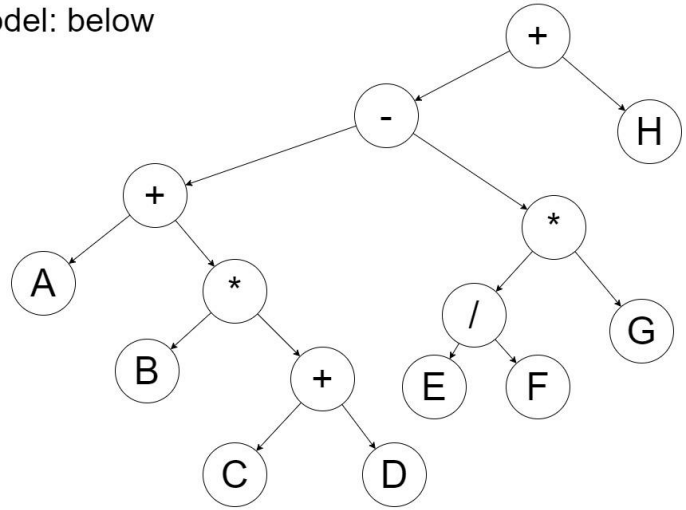
2b conversion:
solution: ABC+D*-EF/+
model: below



2c conversion:
solution: AB+CD-/E+F*G-
model: below

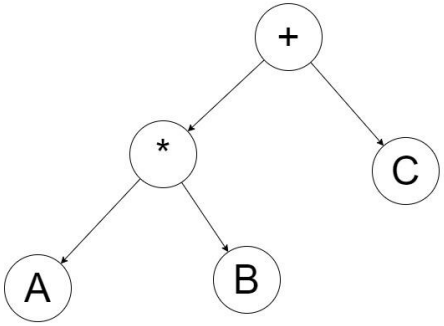


2d conversion:
solution: ABCD+*+EF/G*-H+
model: below

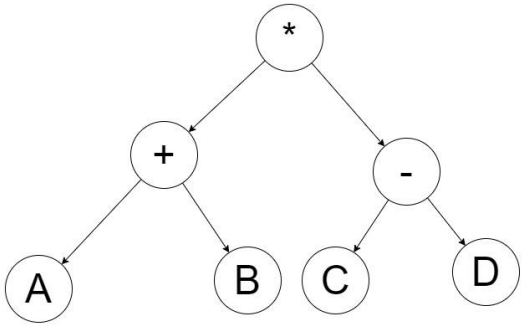


Stacks – Postfix to Infix Solutions

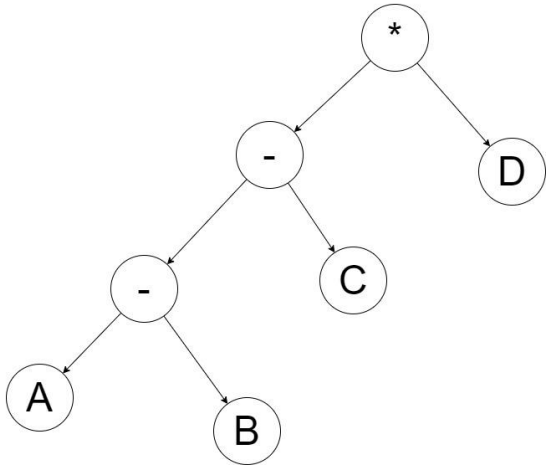
3a conversion:
solution: A*B+C
model: below



3b conversion:
solution: (A+B)*(C-D)
model: below



3c conversion:
solution: (A-B-C)*D
model: below



Assignment

- Read Malik chapter 7 - stacks